Agent001 Assistant for Magento 2

User Guide · module `Agent001_Assistant` · package `agent001/module-assistant` · version 1.0.0

1. Overview

Agent001 Assistant adds an AI shopping assistant to your Magento storefront. The extension:

- connects your Magento installation to your Agent001 account in one click, without copying API keys;
- lets you assign one Agent001 agent to each store view;
- synchronizes the catalog of each store view (products, prices, stock, images, attributes and categories) and keeps it up to date when products and categories change;
- shows the chat widget on every storefront page except the checkout payment pages in store views with an active agent;
- lets the assistant answer order status questions after the customer provides the order number and the e-mail address used for the order.

2. Requirements

Item	Requirement
Magento	Magento Open Source or Adobe Commerce 2.4.6 – 2.4.9
PHP	Any PHP version supported by your Magento release (8.1 or later)
PHP extensions	curl, intl, mbstring
Inventory	Works with Multi-Source Inventory (MSI) and with MSI disabled (legacy CatalogInventory)
Network	HTTPS storefront; the server must be able to reach `https://app.agent001.ai`
Account	An Agent001 account with at least one agent, or the ability to create one during setup
Process	A long-running background worker (`bin/magento agent001:work` , recommended) or the Magento cron, see section 3

3. Installation

Composer (recommended)

```
composer require agent001/module-assistant
php bin/magento module:enable Agent001_Assistant
php bin/magento setup:upgrade
php bin/magento setup:di:compile
php bin/magento setup:static-content:deploy
php bin/magento cache:clean
```

Back up your database before running `setup:upgrade`. In production mode run the commands as part of your usual deployment process, with maintenance mode enabled.

Start the background worker

Catalog synchronization runs in a background worker. Run it under systemd or Supervisor so it restarts automatically, using the same PHP configuration and `app/etc/env.php` as the store:

```
[program:agent001-magento]
command=/usr/bin/php /var/www/magento/bin/magento agent001:work
user=magento
autostart=true
autorestart=true
stopwaitsecs=120
redirect_stderr=true
stdout_logfile=/var/log/agent001-magento.log
```

Keep `autorestart=true` (systemd: `Restart=always`). The worker exits after one hour (`--max-time`, in seconds) or when it uses more than 512 MB of memory and is started again by the process manager, so memory use stays bounded and configuration changes are picked up. Restart the worker after every deployment of new code.

Without a worker, the Magento cron job `agent001_work` (cron group `agent001`, separate process) processes the queue once a minute, so synchronization is slower but does not stop. This also covers environments such as Adobe Commerce Cloud where running custom daemons is restricted.

Nightly full synchronization

The Magento cron job `agent001_nightly` checks every hour and runs a full catalog export once per day at an hour between 01:00 and 04:59 (store time zone). Only the standard Magento cron is required; no additional system cron entry is needed.

4. Setup wizard

Open **Marketing** → **Agent001 Assistant** in the Magento Admin. Access is controlled by the ACL resource **Agent001 Assistant** (`Agent001_Assistant::settings`); grant it to the admin roles that should manage the extension. The wizard has four steps. Your progress is saved automatically, so you can leave the page and continue later.

Step 1 – Connect store

Click **Connect now**. You are redirected to Agent001, where you sign in (or create an account with **Create one**) and approve the connection. You then return to Magento automatically. One connection covers the whole Magento installation; its address is the base URL of the default store view.

Step 2 – Choose agents

For each active store view choose an agent, or leave it without an agent. One agent serves one store view; agents already used by another store view are shown as unavailable. The language of each store view is shown next to its name. Use **New agent** to create an agent in Agent001 for a given store view and **Refresh list** to reload the list afterwards. At least one store view needs an agent. Click **Save and continue**.

Step 3 – Catalog

For each store view with an agent, choose the categories to send. **Select all** (the default) sends the whole catalog, including products without a category and categories added later. **Clear** removes the selection. Selecting a parent category does not select its children automatically. Click **Synchronize catalog** to queue the export. The status of each store view changes from *Queued* to *Reading catalog*, *Indexing* and *Synchronized*. The step completes when all store views with an agent are synchronized. If the worker is not running, the panel shows a warning with the command to start it.

Step 4 – Check chat

Open each store view with the provided links and check that the chat appears in the bottom corner of the page. If you use Varnish or a CDN, purge its cache first. Click **Finish setup**.

5. Dashboard

After the wizard, the same page shows the dashboard:

- **Connection** – the connected Agent001 account and site, with **Reconnect** (renews the installation key and keeps all agents and catalogs) and **Disconnect**.
- **Store views** – one row per store view with the assigned agent, chat status and synchronization status (product count, date of the last synchronization and failed background jobs). Change agents here and click **Save agents**. Changing the agent queues a new synchronization; choosing no agent removes the catalog of that store view from Agent001.

- **Per store view settings** – expand a store view to turn its chat on or off and to change its categories and synchronize the catalog again.

6. Catalog synchronization

What is synchronized

- Enabled products visible in the catalog or search, assigned to the store view's website and salable on its stock (MSI) or in stock (legacy inventory).
- Simple, virtual and downloadable products with their salable quantity (MSI reservations included) or stock quantity; when stock management is disabled the quantity is left empty.
- Configurable products with their salable variants and option attributes. The parent price is the cheapest salable variant.
- Bundle and grouped products with their starting price.
- Prices as calculated by Magento for the store view currency, images (image, small image, thumbnail roles), attributes visible on the storefront, and active categories with their hierarchy.

Keeping the catalog up to date

Saving or deleting a product updates it (and its configurable parents) automatically. When the stock indexer runs in *Update by Schedule* mode, the update is sent after the Magento cron has refreshed the index, so make sure the Magento cron is running. Product imports, mass attribute updates and category changes queue a full export. Changes that Magento does not report through events (for example price rules or stock reservations) are picked up by the nightly synchronization.

Console commands

Command	Purpose
`bin/magento agent001:work`	Runs the background worker continuously.
`bin/magento agent001:work --max-time=<seconds>`	Exits after the given time (default 3600) so the process manager restarts it.
`bin/magento agent001:work --once`	Processes up to 20 pending jobs and exits.
`bin/magento agent001:work --retry-failed --once`	Retries failed jobs of all store views.
`bin/magento agent001:sync`	Queues a full export of all store views with an agent.
`bin/magento agent001:sync --store=<code|id>`	Queues a full export of one store view.
`bin/magento agent001:sync --due`	Queues the nightly export when its hour has come (the `agent001_nightly` cron job does this automatically).

Failed jobs are retried up to five times with an increasing delay. Remaining failures are shown in the dashboard.

Limits

A full synchronization sends up to 50,000 products per store view and must finish within 2 hours. Larger catalogs: limit the categories in step 3 or contact support.

7. Chat widget

The widget is added to the `before.body.end` container of every storefront page and works with the full page cache and Varnish. It is shown only in store views whose agent is active and whose first synchronization has finished. Turning a store view's chat off or disconnecting the store removes the widget. Your theme must render the `before.body.end` container; headless or PWA storefronts need the loader embedded separately.

The widget script is not loaded on the payment pages of Magento checkout: one-page checkout (`checkout_index_index`) and multishipping billing (`multishipping_checkout_billing`). Third-party checkout modules with their own layout handle (for example `onestepcheckout_index_index`) are not covered. Remove the widget there with a layout file of that handle in your theme, otherwise list the loader in your payment page script inventory (PCI DSS 4.0, requirements 6.4.3 and 11.6.1):

```
<page><body><referenceBlock name="agent001.widget" remove="true"/></body></page>
```

8. Order status lookup

When a customer asks about an order, the assistant asks for the order number and the e-mail address used for the order. The extension returns only the order status, dates, totals, shipping method, tracking numbers and item names, quantities and amounts. It never returns names, addresses, phone numbers, e-mail addresses or payment data. A wrong number or e-mail address returns the same "not found" answer. Lookups are accepted only as POST requests with a JSON body, so the e-mail address does not appear in URLs or access logs. Chat lookups are limited to 20 per minute per visitor IP. After 5 failed attempts an e-mail address is blocked for one hour, and so is an order number for the visitor IP that tried it. The visitor IP is resolved with Magento's `RemoteAddress` . Behind Varnish, a CDN or a load balancer make sure the real client IP reaches Magento, for example with the nginx `real_ip` module or the `alternativeHeaders` and `trustedProxies` arguments of `RemoteAddress` in `di.xml` .

9. Configuration options

No configuration is needed for the standard Agent001 service. Advanced installations can set the following keys in `app/etc/env.php` :

```
'agent001' => [
    'base_url' => 'https://app.agent001.ai',
    'internal_base_url' => 'https://app.agent001.ai',
    'allowed_origins' => ['https://another-approved.example'],
],
```

`internal_base_url` is used for server-to-server calls and `allowed_origins` lists additional origins allowed to call the order lookup. The extension declares the Content Security Policy entries for the default Agent001 domain.

10. Data and privacy

The extension sends public catalog data to Agent001: product names, descriptions, SKUs, prices, availability, product and image URLs, attributes and categories, and the store URL. Customer data is not sent. Order data is returned only for a single order after the order number and e-mail address are verified. The Agent001 keys are stored encrypted with the Magento encryption key and never appear in storefront HTML.

11. Troubleshooting

Symptom	Solution
Status stays at *Queued* and the panel shows a worker warning	Start `bin/magento agent001:work` and check that it keeps running, or check that the Magento cron is running.
Failed background jobs are shown	Check that the server can reach Agent001, then run `bin/magento agent001:work --retry-failed --once`.
The chat does not appear	Check that the store view has an active agent and is synchronized, purge Varnish or CDN caches, and check the browser console for Content Security Policy errors.
Agents cannot be loaded	Agent001 is unreachable or the connection was removed in Agent001. Click **Reconnect**.
The default store view base URL changed	Disconnect the store and connect it again.
A large category tree cannot be saved	Increase the PHP `max_input_vars` limit.

12. Uninstalling

1. Click **Disconnect** in the dashboard. This removes the store connection and its catalogs from Agent001; your Agent001 account and agents stay.
2. Stop the worker.

3. Run `composer remove agent001/module-assistant`, then `bin/magento setup:upgrade` and `bin/magento cache:clean`.

13. Support

For help, use the contact form at agent001.ai/contact or write to biuro@okinet.pl. Agent001 is provided by Okinet s.c., ul. Sucha 3, 50-086 Wrocław, Poland.